The Code Length of Post-Trained Language Models on Fresh Text

Musaab Mahjoub
Independent researcher

October 2026

Abstract

A method is described for measuring how far post-training moves a language model from the distribution on which it was pretrained: the code length, in bits per byte, that the model assigns to text written after its training cutoff, compared with the code length its base model assigns to the same text. We measured this quantity for 12 public post-trained checkpoints of 0.5 to 3 billion parameters and for 12 checkpoints of our own fine-tuning runs, and compared it with benchmark accuracy. Every post-training stage lengthened the code, by amounts from something of the order of 0.1% for pure reinforcement learning to 43% for mathematical training followed by reasoning distillation, and one temperature removes at most a third of the increase. The increase measures how much post-training was applied: it tracks the instruction-following gain, and above 15% it was always accompanied by forgetting, but when the increase is held fixed the recipe alone decides whether a model forgets.

1. Introduction

The entropy of a language can be estimated from the number of bits a good predictor needs to encode its text [2, 18, 19]. A language model is such a predictor, and its code length on unseen text has been proposed as a measure of what it has learned [4, 7, 12]. Huang et al. [7] found, across 31 base models and 12 benchmarks, that the average benchmark score is close to a linear function of the average bits per character on held-out text, with a Pearson correlation of -0.93, and of about -0.95 within each domain. They excluded fine-tuned models, on the ground that such a model no longer predicts arbitrary text but only structured query and response data. Most models in use are fine-tuned, and what their code length on general text measures has been left open.

In the present paper we measure the code length of post-trained models on general text written after their training cutoff, relative to that of their base models on the same text. We call the relative increase the *drift*. Three questions are taken in order: whether post-training changes the code length, by how much and at which stage; whether the change is a matter of calibration that one temperature can undo; and whether the change predicts what post-training cost in general benchmark accuracy while ignoring what it gained. The third question is the practical one. The drift of a checkpoint costs one forward pass over 300 kilobytes of text, a median of 19 s on one GPU, while one pass of our benchmark suite costs a median of 18 min, 67 times as much. If the cheap number predicted the expensive one, a sweep of post-training recipes could be screened at a small fraction of its evaluation cost.

The answers are the following.

1. Post-training lengthened the code in all 15 comparisons we made, at every stage and on each of three fresh corpora. Pure reinforcement learning moved it by 0.1–0.5%, supervised fine-tuning (SFT) and preference optimisation (DPO) by 1–7% per step, and mathematical training with reasoning distillation by 16–43% (Section 4.1).
2. A single temperature recovers at most 35% of the increase. The rest is a change in what the model predicts, not in how confident it is (Section 4.2).
3. Across the 12 public checkpoints the drift correlates with forgetting at $r = 0.89$, but the correlation rests on two points. The data are a threshold: every checkpoint above 15% forgot, while 8 of the 12 lie within the benchmark noise. The drift tracks the instruction-following gain as closely ($r = 0.89$), so it is not a one-sided meter of loss (Section 4.3).
4. The drift is one-dimensional across token classes. A contrast between knowledge tokens and style tokens gave a lead, $r = 0.81$ with forgetting where the total drift gave 0.32 (Section 4.4).

5. Under recipe control the lead fails. With one base and three SFT recipes compared at equal drift, forgetting is a step of about 4% set by the recipe at the first checkpoint and constant thereafter, and neither the total drift nor its contrast tells the recipes apart (Section 4.5).

Each experiment was pre-registered: the rules and expected values were committed before the runs, and Appendix A sets them beside the outcomes. Code, dated corpora, per-token bits and benchmark outputs are in the repository named in Appendix B.

2. Definitions

Let $x = x_1 \cdots x_n$ be a text, encoded as a sequence of tokens, and let q be a language model, which assigns to each token a conditional probability $q(x_i \mid x_{<i})$.

Definition. The *code length* of x under q is $L_q(x) = -\sum_{i=1}^{n} \log_2 q(x_i \mid x_{<i})$, in bits. Divided by the number of UTF-8 bytes of x it is the code length per byte, in bits per byte (bpb).

For example, Qwen2.5-1.5B codes the 101,843 bytes of our Wikipedia corpus in 73,191 bits, or 0.719 bpb. It is a code length in the strict sense: an arithmetic coder driven by q transmits x in $L_q(x)$ bits plus a small constant [4].

Suppose the text was produced by a source with distribution p. The expected code length of one token is then the sum of two terms [18],

$$\mathbb{E}_p\big[-\log_2 q(X_i \mid X_{<i})\big] = H_p(X_i \mid X_{<i}) + \mathbb{E}_p\, D\big(p(\cdot \mid X_{<i}) \,\big\|\, q(\cdot \mid X_{<i})\big), \tag{1}$$

where H_p is the conditional entropy of the source and D is the Kullback–Leibler divergence of the model from the source, both in bits. The first term depends only on the source and the second only on how far the model is from it. It is easily seen from (1) that when a post-trained model m and its base b, with identical tokenizers, code the same text, the entropy of the text cancels from the difference:

$$\mathbb{E}_p\big[L_m(X) - L_b(X)\big] = \sum_{i=1}^{n} \mathbb{E}_p\Big[D\big(p(\cdot \mid X_{<i}) \,\big\|\, q_m(\cdot \mid X_{<i})\big) - D\big(p(\cdot \mid X_{<i}) \,\big\|\, q_b(\cdot \mid X_{<i})\big)\Big]. \tag{2}$$

The difference of code lengths on one text therefore estimates how much further from the source of that text the post-trained model has moved, in bits, with the unpredictability of the text removed.

Definition. The *drift* of a model m from its base b on a corpus x is $\Delta = 100\,(L_m(x) - L_b(x))/L_b(x)$, in per cent. Because both models code the same bytes, Δ is also the relative change in bits per byte. The drift on fresh text, Δ_{fresh}, is the unweighted mean of Δ over the three corpora written after the models were released (Section 3.1).

The base is the root of the model's family, the pretrained checkpoint from which it descends. Where a checkpoint has an intermediate parent we also report the step against the parent. Qwen2.5-1.5B-Instruct, for example, codes the Wikipedia corpus in 73,674 bits, a drift of +0.66% from Qwen2.5-1.5B.

Definition. The *paired window bootstrap* gives the interval of a drift. Each corpus is cut into consecutive windows of at most 1,024 tokens, never splitting a character, and each window is scored after one document-start token. The models of a family see identical windows. Whole windows are resampled with replacement, the same windows for both models, 5,000 times, and the 2.5 and 97.5 percentiles of Δ form the 95% interval. For Δ_{fresh} the windows are resampled within each corpus separately.

The interval measures the variation due to the sample of text. It does not contain the variation between training runs, which one public checkpoint per stage cannot show.

Definition. The *token-class drift* Δ_c is the drift computed on the bits of the tokens of class c alone. Each token takes the part of speech of the word under its first non-space character (spaCy `en_core_web_sm`). The class KNOWLEDGE holds proper nouns, numbers, and nouns, adjectives and verbs outside the 2,000 most frequent English words; STYLE holds function words, punctuation and whitespace; the rest is MIDDLE. The *contrast* is $K - S = \Delta_{\text{KNOWLEDGE}} - \Delta_{\text{STYLE}}$, in percentage points (pp). A frequency-only variant replaces the classes by token ids below and above the median count in the corpus (rare − frequent).

For example, OLMo-2-1B-DPO lengthens the code of its knowledge tokens by 13.20% and that of its style tokens by 16.20%, a contrast of −3.00 pp.

The drift has the following limits, which follow from its definition.

1. It is not the divergence between the post-trained model and its base. Equation (2) compares each model with the source of the text; two models can differ widely and be equally far from it.
2. It is a property of the text as well as of the model. Qwen2.5-Math-1.5B is +5.7% from its base on arXiv abstracts and +33.9% on Wikipedia.
3. It is measured on raw text. Scored inside both models' chat templates, Qwen3-1.7B is +37.0% from its base, against +16.2% on raw text.
4. It is defined only between models whose tokenizers produce identical token ids on the corpus.
5. It is a distance, not a capability score, and Section 4 shows that a distance which costs capability and one which does not can be of the same size.

3. Materials

3.1 Corpora

Three corpora of new text and one control were collected as dated snapshots, each with a provenance file listing the URL and date of every document (Table 1). Each holds 20,000 to 35,000 tokens, or 21 to 34 windows. The models studied were released in 2024 and 2025 [1, 3, 17, 21, 22, 25], and the fresh documents were published in October 2026.

Table 1. The corpora.

Corpus	Bytes	Docs	Dates	Content
wiki2026	101,843	17	created 5–13 Oct 2026	New Wikipedia articles: 6 pages on 2026 events and 11 other new articles; references removed, at most 12,000 characters each
arxiv2026	100,041	80	published 9 Oct 2026	Title and abstract of 40 cs.LG and 40 math.AP papers, interleaved
gutenberg2026	106,023	4	released 12–13 Oct 2026	About 26 KB from each of four Project Gutenberg books, past the front matter; new transcriptions of books from the 1900s to 1920s
enwik8_ctrl	100,000	1	2006	Bytes 1,000,000–1,100,000 of enwik8 (Wikipedia with markup), a control that every model has probably seen

3.2 Checkpoints

We scored 18 public checkpoints in six families: six bases and 12 post-trained descendants (Table 2). They use four tokenizers (OLMo-2, Qwen2.5, Qwen3, SmolLM2), and within each tokenizer group every model encodes all four corpora to identical token ids, which we verified. The OLMo-2 family is a chain of stages, base, SFT, DPO and RLVR [11, 21]. Qwen2.5-1.5B branches into its Instruct model, a pure-RL model trained directly from the base, and the mathematical line on which R1-Distill was trained [3, 17, 25]. Both Qwen2.5-Math-1.5B and R1-Distill are compared with plain Qwen2.5-1.5B as their family base.

3.3 Controlled recipes

To separate the recipe from the dose we fine-tuned one base, OLMo-2-0425-1B, with three SFT recipes that differ only in their data: *tulu* (the Tülu 3 SFT mixture, OLMo-2's own SFT data [11]), *smoltalk* (SmolTalk, the SFT data of SmolLM2-Instruct [1]) and *mathcot* (one DeepSeek-R1 reasoning answer per problem from OpenR1-Math-220k [3, 15]). Each recipe is a subset of 45.0 million template tokens taken in a fixed random order: 79,249 examples of mean length 568 tokens for *tulu*, 59,959 of 751 for *smoltalk* and 30,512 of 1,475 for *mathcot*. Examples longer than 2,048 tokens were dropped, which biases *mathcot*

Table 2. The 12 post-trained checkpoints, grouped by family base. Recipes as described by the model publishers.

Family base	Checkpoint	Stage	Recipe
OLMo-2-0425-1B	-SFT	SFT	Tülu 3 SFT mixture
	-DPO	DPO	Tülu 3 on-policy preference DPO, on top of SFT
	-Instruct	RLVR	GRPO on verifiable math and instruction rewards, on top of DPO
Qwen2.5-0.5B	-Instruct	instruct	Qwen2.5 pipeline: SFT on about 1M examples, offline DPO, online GRPO
	SimpleRL-Zoo	pure RL	GRPO from the base on GSM8K and MATH problems, rule-based reward
Qwen2.5-1.5B	-Instruct	instruct	Qwen2.5 pipeline
	SimpleRL-Zoo	pure RL	as for 0.5B
	Qwen2.5-Math-1.5B	math	continued pretraining on the Qwen Math Corpus v2
	R1-Distill-Qwen-1.5B	distill	SFT on about 800k DeepSeek-R1 reasoning traces, on top of Qwen2.5-Math-1.5B
Qwen2.5-3B	-Instruct	instruct	Qwen2.5 pipeline
SmolLM2-1.7B	-Instruct	instruct	SFT on SmolTalk, then DPO on UltraFeedback
Qwen3-1.7B-Base	Qwen3-1.7B	instruct	strong-to-weak distillation from the Qwen3 flagship models

toward short solutions. All examples were rendered with the OLMo-2-Instruct chat template and trained with the loss on assistant tokens only.

Training was a full fine-tune with fp32 master weights and bf16 autocast, AdamW at a peak learning rate of 2×10^{-5}, 32 packed rows of 2,048 tokens per update and seed 0. The first stage ran 1,500 updates (98.3 million token positions, 1.6 to 1.7 passes over the subset) with a cosine schedule. No recipe reached 6% drift in that budget, and a pre-registered second stage continued each run from its final checkpoint with a fresh optimiser and a new warm-up, until a wall-time limit stopped it at updates 3,464 to 3,489 (3.8 to 4.0 passes). The drift was scored every 20 updates against the base on the same GPU, and a checkpoint was saved at the first scored update at or above 3%, 6% and 10%. The 10% mark was never reached. With the final checkpoint of each stage this gives 12 checkpoints, six of them at marks.

3.4 Benchmarks

All checkpoints and bases were evaluated with lm-evaluation-harness 0.4.13 [5] in fp16 on A10 GPUs, with greedy decoding at the harness defaults. The general score g is the mean of six zero-shot accuracies: ARC-Easy, ARC-Challenge, HellaSwag (first 2,000 documents), PIQA, WinoGrande and MMLU (first 40 questions of each of 57 subjects).

Definition. The *forgetting* of a checkpoint is $F = 100\,(g_{\text{base}} - g)/g_{\text{base}}$, in per cent of the base score; a positive value is a loss.

The noise floor of a difference between two checkpoints is $1.96\sqrt{\text{se}_1^2 + \text{se}_2^2}$, from the harness's standard errors of the six-task means, expressed in per cent of the base score. It is about 2.1% (1.9–2.5%) for the public checkpoints and 2.2% for our own. The target tasks are GSM8K (first 300 problems, five-shot, generation capped at 256 tokens) and IFEval (first 200 prompts, prompt-level strict accuracy, capped at 1,280 tokens). The gain of a chat-tuned model was measured with its own chat template, against its base prompted without one.

3.5 Scoring and its checks

Every text token is scored, with $-\log_2 p$ taken from the float32 log-softmax of one forward pass per window. The document-start token is the tokenizer's beginning-of-sequence token where the tokenizer adds one (R1-Distill only) and <|endoftext|> otherwise. The public checkpoints were scored with fp16 weights, on a laptop GPU (Apple M2 Pro) for the SmolLM2 and Qwen3 pairs and on a Tesla T4 for the rest; our own runs were scored in bf16 on the training GPU. The mechanics were checked in six ways.

1. The summed bits equal the library's cross-entropy loss times the token count divided by $\ln 2$, for every model, to within 10^{-7} relative.

2. fp16 and fp32 agree on the first window of every model to within 1.9×10^{-4} relative; nine windows with extreme surprisals, rescored in fp32, differ by at most 5.7×10^{-4}.
3. Repeated forward passes are bit-identical.
4. The four models scored on both the laptop and the T4 agree to within 5×10^{-5} bpb on all 16 rows.
5. GPT-2 (124M) codes the enwik8 control at 1.243 bpb.
6. In our own runs the bf16 base agrees with the fp16 score of the same base to within 2.2×10^{-4} relative on all four corpora.

4. Results

4.1 Post-training lengthens the code at every stage

Table 3 gives the drift of every post-trained checkpoint from its family base, and Fig. 1 shows it on a logarithmic scale. Post-training lengthened the code in every case. Of the 15 comparisons, 12 against the family base and 3 against an intermediate parent, each has its 95% interval above zero on each of the three fresh corpora separately, and no reinforcement-learning stage shortened the code on any corpus.

Table 3. Drift on fresh text, Δ_{fresh} in per cent, with 95% paired window-bootstrap intervals. The step column gives the drift from the parent stage where the parent is not the base. The last column is the drift on the old enwik8 control.

Family	Checkpoint	Δ_{fresh}	95% interval	Step from parent	enwik8
OLMo-2-1B	SFT	+6.99	[6.61, 7.40]		+9.35
	DPO	+14.10	[13.52, 14.69]	+6.65 [6.38, 6.93]	+19.23
	RLVR (Instruct)	+14.66	[14.05, 15.28]	+0.49 [0.44, 0.55]	+19.66
Qwen2.5-0.5B	Instruct	+1.61	[1.49, 1.73]		+2.76
	SimpleRL-Zoo	+0.18	[0.16, 0.21]		+0.40
Qwen2.5-1.5B	Instruct	+0.91	[0.81, 1.01]		+1.58
	SimpleRL-Zoo	+0.12	[0.09, 0.14]		+0.28
	Qwen2.5-Math	+23.84	[22.46, 25.41]		+58.72
	R1-Distill	+42.95	[41.07, 44.93]	+15.81 [14.35, 17.29]	+74.30
Qwen2.5-3B	Instruct	+2.08	[1.74, 2.47]		+6.18
SmolLM2-1.7B	Instruct	+3.57	[3.25, 3.87]		+12.51[a]
Qwen3-1.7B	Qwen3-1.7B	+16.18	[15.67, 16.73]		+24.93

[a] An artefact of the document-start token on two enwik8 windows; with a blank line as document start the enwik8 drift of this pair is +7.9%.

The size of the increase spans more than two orders of magnitude, and it is ordered by the kind of training. The two pure-RL checkpoints moved the code by 0.18% and 0.12%, 8.7 and 7.8 times less than the Instruct pipelines of the same sizes, and OLMo-2's RLVR step moved it by 0.49%, fourteen times less than its SFT step. The SFT and DPO steps moved it by 1 to 7%, and OLMo-2's DPO step (+6.65%) was as large as its SFT step (+6.99%), against the 0 to 2% we had expected. The largest increases belong to the mathematical line: continued pretraining on mathematics moved Qwen2.5-1.5B by 23.84%, and the distillation of R1 reasoning traces on top of it added 15.81%, for 42.95% in all.

The signs do not depend on the scoring convention. With a blank line as document start the four pairs of Table 3 scored on the laptop (Qwen2.5-1.5B, SmolLM2, Qwen3 and Math to R1-Distill) give +0.9, +4.1, +17.0 and +20.9%; restricted to tokens with at least 128 tokens of context they give nearly the same deltas. The extra bits are spread over the text. For Qwen3 and R1-Distill the worst 1% of tokens carry 11 to 19% of them, and the post-trained model is worse than its base on only 43 to 63% of the tokens: it wins a little on easy tokens and loses a great deal on surprising ones.

The old control drifts more than the fresh text. For every stage scored on the T4 the enwik8 drift is 1.3 to 3.0 times the fresh drift, the one exception being R1-Distill against its parent (0.6). The increase is therefore not specific to text written after the cutoff, a point to which Section 4.3 returns.

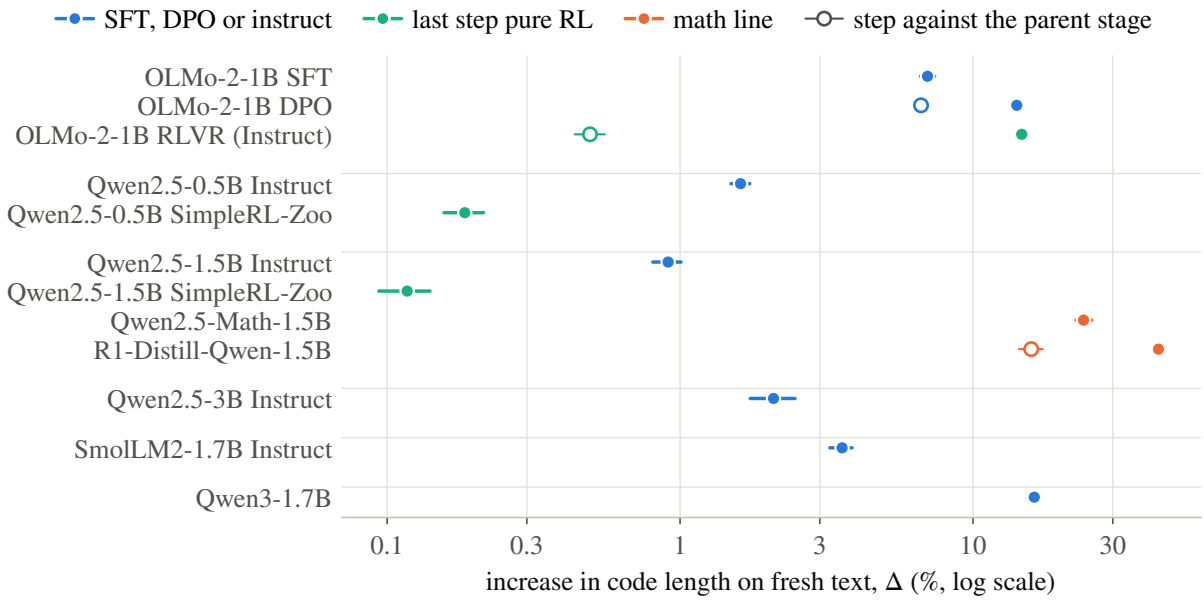


Fig. 1. Drift of each post-trained checkpoint on fresh text, against its family base (filled) and, for chained stages, against its parent stage (hollow). Lines are 95% paired window-bootstrap intervals; some are narrower than the marker. The colour gives the kind of the last training step.

4.2 One temperature recovers at most a third

If post-training only sharpened the predicted distributions, a single temperature T applied to the post-trained model's logits would restore most of the code length. For the four laptop pairs we fitted the best temperature T^* on the even windows of each corpus and measured on the odd windows. The rule, stated before the run, called the increase mostly a confidence shift if both large-drift pairs recovered at least 70% of it, and mostly a change of belief if both recovered at most 30%. Table 4 and Fig. 2 give the outcome.

Table 4. Temperature fit on the odd windows of the fresh corpora. Recovered is the share of the increase at $T = 1$ removed by the post-trained model's own T^* (mean of the three corpora; the range over corpora in parentheses). Residual is the drift with each model at its own T^*.

Pair	Δ at $T = 1$	T^* base	T^* post	Recovered	Residual
Qwen2.5-1.5B → Instruct	+1.0%	1.00–1.05	1.00–1.06	15% (1–42)	+0.9%
SmolLM2-1.7B → Instruct	+3.7%	1.00–1.05	1.04–1.08	6% (2–13)	+3.6%
Qwen3-1.7B-Base → Qwen3-1.7B	+16.0%	1.00–1.04	1.22–1.28	35% (31–39)	+10.4%
Qwen2.5-Math-1.5B → R1-Distill	+15.4%	1.02–1.07	1.15–1.18	19% (18–20)	+12.7%

Qwen3 and R1-Distill are overconfident on fresh text. Their best temperatures are about 1.25 and 1.16, while every base model's lies between 0.995 and 1.07. Undoing the sharpening removes 35% of Qwen3's increase (per-corpus intervals 28 to 42%) and 19% of R1-Distill's. With each model at its own best temperature the post-trained model still needs 10.4% and 12.7% more bits than its base. Under the stated rule the verdict is mixed: Qwen3 falls in the middle band and R1-Distill in the band of changed belief. For the two light instruction-tuned pairs the best temperature is within 0.04 of the base's, and it recovers 6 and 15%. The result agrees with Yang [23], who found on web text that a temperature together with a vocabulary bias recovers at most 36% of the increase in loss. One global temperature is the crudest calibrator, and a miscalibration that depends on the context would appear here as changed belief.

4.3 What the drift predicts on benchmarks

We benchmarked the 12 post-trained checkpoints and their six bases. The pre-registered gate was a Pearson correlation of at least 0.8 between Δ_{fresh} and forgetting over the 12; a second rule, for one-sidedness, required the correlation with the gain to stay below 0.3 in magnitude. Table 5 and Fig. 3 give the outcome.

The gate passes as written: $r = 0.887$, with a bootstrap interval of $[-0.01, 0.98]$ over checkpoints.

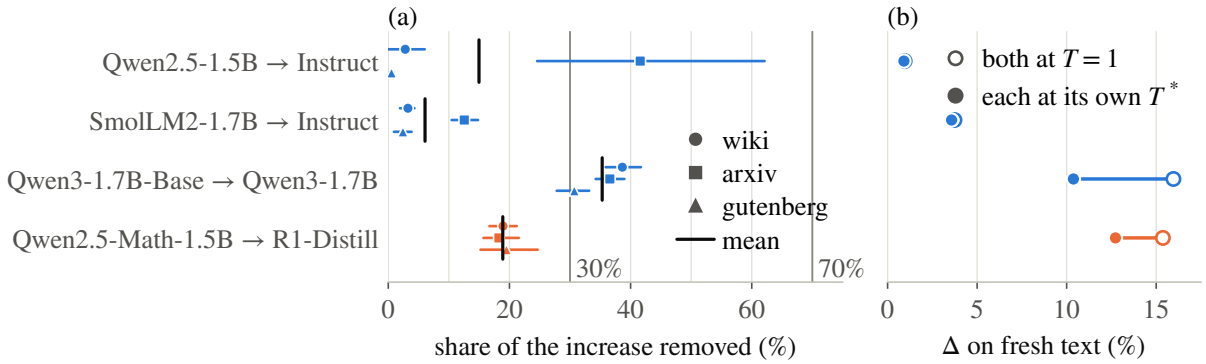


Fig. 2. (a) Share of the increase in code length removed by the post-trained model's own best temperature, per fresh corpus, with 95% window-bootstrap intervals; the black bar is the mean over the three corpora and the grey lines are the pre-registered 30% and 70% boundaries. (b) The drift with both models at $T = 1$ (hollow) and with each at its own T^* (filled), mean of the three fresh corpora.

Table 5. Drift, forgetting and gain of the 12 public checkpoints. The general score is the mean of six accuracies in per cent; the floor is the 95% noise floor of the forgetting; the IFEval gain is in points, with each post-trained model in its own chat template.

Checkpoint	Δ_{fresh}	General, base → post	Forgetting	Floor	IFEval gain
OLMo-2-1B SFT	+6.99	58.89 → 58.88	+0.01	2.2	+31.5
OLMo-2-1B DPO	+14.10	58.89 → 59.71	−1.40	2.2	+43.5
OLMo-2-1B RLVR	+14.66	58.89 → 59.41	−0.88	2.2	+50.5
Qwen2.5-0.5B Instruct	+1.61	52.74 → 52.85	−0.22	2.5	+13.0
Qwen2.5-0.5B SimpleRL-Zoo	+0.18	52.74 → 52.82	−0.15	2.5	+5.5
Qwen2.5-1.5B Instruct	+0.91	62.69 → 63.81	−1.78	2.1	+18.0
Qwen2.5-1.5B SimpleRL-Zoo	+0.12	62.69 → 63.67	−1.57	2.1	−6.0
Qwen2.5-Math-1.5B	+23.84	62.69 → 53.97	+13.90	2.1	+1.5
R1-Distill-Qwen-1.5B	+42.95	62.69 → 49.14	+21.61	2.1	+5.5
Qwen2.5-3B Instruct	+2.08	66.27 → 66.56	−0.44	1.9	+34.0
SmolLM2-1.7B Instruct	+3.57	62.68 → 60.30	+3.79	2.1	+29.5
Qwen3-1.7B	+16.18	62.54 → 59.62	+4.67	2.1	+46.5

The pass rests on two points. Qwen2.5-Math and R1-Distill drift by 24 and 43% and lose 13.9 and 21.6% of their base score; without them $r = 0.32$ over the remaining ten. Spearman's ρ is 0.67 over all 12 and 0.43 over the ten, and the leave-one-out range of r is 0.69 to 0.93. The width of the interval is honest: with 12 checkpoints the pass depends on two of them. Chance-adjusted forgetting, the aggregate form of the adjustment of Harmon et al. [6], ranks the 12 checkpoints identically.

We therefore read the data as a threshold and not as a ranking. Eight of the 12 checkpoints lie inside their noise floor, among them OLMo-2-DPO and -RLVR at drifts of 14.1 and 14.7%. All three checkpoints above 15% lie outside it: Qwen3 at 16.2% (forgetting 4.7%), Qwen2.5-Math and R1-Distill. One checkpoint below the threshold forgets: SmolLM2-1.7B-Instruct loses 3.8% at a drift of 3.6%, nearly all of it on ARC-Easy, which falls by 10.6 points. Two SFT and DPO recipes thus land on opposite sides of the meter, OLMo-2's at 14% drift with no loss and SmolLM2's at 3.6% with a loss. R1-Distill is not a case of large drift with modest loss: against its own parent it drifts 15.8% and loses another 9.0%.

Old text predicts as well as new. The enwik8 drift gives $r = 0.949$ and $\rho = 0.678$ against forgetting. For this outcome the freshness of the text adds nothing we could measure.

The drift is not a one-sided meter. The pre-registered gain, GSM8K prompted without chat templates, cannot be read for chat-tuned models: SmolLM2-Instruct scores 0.0 when prompted as a base model. With chat templates the GSM8K gain is confounded by the 256-token generation cap, which ends R1-Distill's reasoning before its answer (15.7%), and its correlation with the drift ($r = -0.47$) cannot be interpreted. IFEval has a cap of 1,280 tokens and no reasoning model among the eight chat-tuned checkpoints. On it the drift tracks the gain as closely as it tracks forgetting: $r = 0.89$ (interval $[0.70, 0.99]$), $\rho = 0.88$.

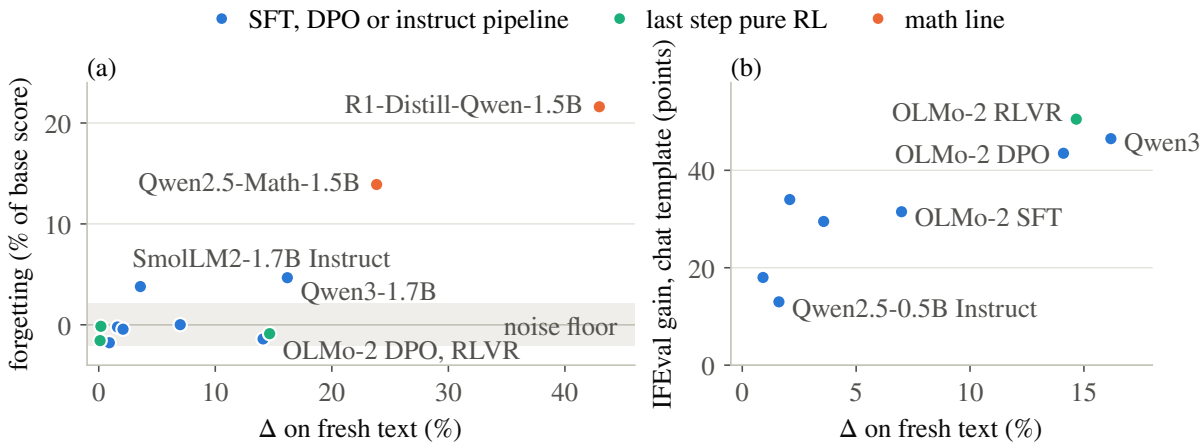


Fig. 3. (a) Forgetting against drift for the 12 public checkpoints; the grey band is the median noise floor, ±2.1% of the base score. (b) IFEval gain, each post-trained model in its own chat template, against drift for the eight chat-tuned checkpoints (stages SFT, DPO, RLVR and instruct).

OLMo-2 SFT, DPO and RLVR, at drifts of 7, 14 and 15%, gain 31.5, 43.5 and 50.5 points, and Qwen3 at 16% gains 46.5. The drift measures how much post-training was applied. A larger dose bought more instruction following, and in these checkpoints a dose above 15% also cost general accuracy.

4.4 The token-class decomposition

If the drift had two components, one on names, numbers and rare words that follows lost knowledge and one on function words and punctuation that follows changed style, the first might predict forgetting and the second the gain. We split the drift of the same 12 checkpoints by token class (Section 2), without new scoring. The pre-registered fail rule was a correlation above 0.9 between knowledge drift and style drift, which would make the drift one-dimensional.

The rule triggers: $r(\Delta_K, \Delta_S) = 0.925$, interval [0.72, 1.00]. Knowledge drift ranks the 12 checkpoints exactly as total drift does ($\rho = 0.671$ for both, against forgetting), and both class drifts track the IFEval gain ($r = 0.84$ and 0.81), so neither channel hypothesis passes. Two case predictions pass: SmolLM2-Instruct puts 63.0% [60.1, 66.2] of its extra bits on knowledge tokens and OLMo-2-DPO 44.7% [42.6, 46.8]. Knowledge tokens take more than their proportional share of the extra bits in 9 of the 12 checkpoints.

One secondary reading, registered without a threshold, gave a lead. The contrast $K - S$ tracked forgetting with $r = 0.92$ over the 12 and $r = 0.81$, interval [0.41, 0.98], without the two mathematical checkpoints, where the total drift gave 0.32. It did not track the IFEval gain ($r = 0.17$ over the eight). The frequency-only variant, which needs no tagger, gave 0.82, 0.74 and −0.07. Two checkpoints at the same dose show the reading most plainly. OLMo-2-DPO drifts 14.1% with a contrast of −3.00 pp and forgets nothing; Qwen3 drifts 16.2% with a contrast of +14.12 pp and forgets 4.7%. The lead rested on about four informative checkpoints from different laboratories, bases and recipes, and it called for a test in which the base and the dose are held fixed.

4.5 The matched-recipe test

The test holds the base and the dose fixed and varies the recipe (Section 3.3). At a drift mark the three recipes have equal Δ_{fresh} by construction: 3.02 to 3.28% at the 3% mark and 6.00 to 6.06% at the 6% mark. If forgetting still differs between them, the contrast must account for it. Table 6 and Fig. 4 give the 12 checkpoints.

The pre-registered fail rule triggers. Over the six mark checkpoints the correlation of the contrast with forgetting is $r = 0.43$, interval [−0.79, 1.00], below the threshold of 0.6; its correlation with the IFEval gain is $r = -0.41$, outside the required ±0.3; and at the 6% mark the contrast orders the recipes *mathcot* 3.41 ≈ *tulu* 3.37 > *smoltalk* 2.38 pp, where *mathcot* > *smoltalk* > *tulu* was predicted. Over all 12 checkpoints the first correlation is 0.29. At the 6% mark *tulu* has the contrast of *mathcot* and forgets nothing. Within each recipe the contrast grows with the dose ($r = 0.80$ with Δ_{fresh} over the 12), so it is in

Table 6. The 12 checkpoints of the controlled recipes. Passes are passes over the 45M-token subset. The IFEval gain is in points against the base in the same template; ARC-e and MMLU are accuracies (base 0.735 and 0.383).

Recipe	Update	Mark	Passes	Δ_{fresh}	$K-S$ (pp)	Forgetting	IFEval gain	ARC-e	MMLU
tulu	1500	final	1.72	+2.02	+1.69	−1.45	+14.0	0.730	0.418
	1760	3%	2.02	+3.28	+2.47	−0.30	+15.5	0.707	0.438
	2660	6%	3.05	+6.01	+3.37	−1.74	+13.5	0.726	0.439
	3489	final	4.00	+7.11	+4.35	−0.82	+12.0	0.714	0.431
smoltalk	920	3%	1.02	+3.02	+1.55	−0.97	+10.5	0.722	0.426
	1500	final	1.67	+3.36	+2.17	−0.83	+13.0	0.717	0.432
	2500	6%	2.78	+6.00	+2.38	−0.89	+13.5	0.719	0.428
	3487	final	3.87	+8.97	+3.41	−0.65	+16.5	0.707	0.429
mathcot	1500	final	1.63	+2.50	+2.10	+4.24	+1.5	0.641	0.371
	1660	3%	1.81	+3.06	+2.86	+4.07	+3.0	0.638	0.375
	2780	6%	3.02	+6.06	+3.41	+4.10	+0.0	0.662	0.363
	3464	final	3.77	+7.14	+4.48	+4.41	+3.5	0.663	0.366

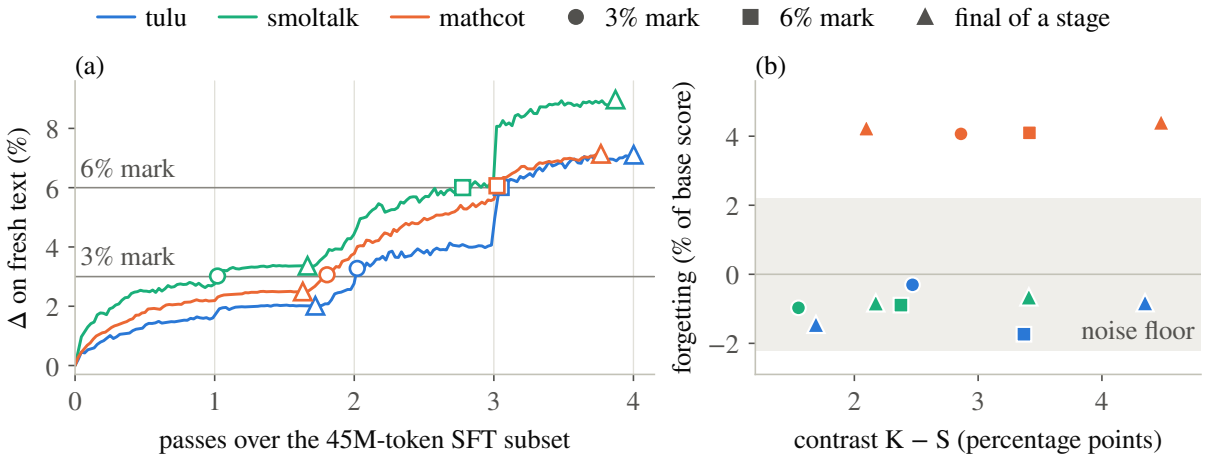


Fig. 4. The controlled recipes on one base. (a) Drift on fresh text along each run, against passes over the training subset; markers are the benchmarked checkpoints, and the first triangle of each run, at 1.6 to 1.7 passes, is where the second stage restarted. (b) Forgetting against the contrast $K-S$ for the 12 checkpoints; the grey band is the noise floor, ±2.2% of the base score.

large part the total drift seen through a subset of tokens. The rare − frequent variant does a little better over the marks ($r = 0.69$), but it holds its predicted order by a margin of 0.07 pp and gives 0.35 over all 12.

What the run shows is that the recipe sets forgetting as a step. *mathcot* forgets 4.07 to 4.41% of the base score at every checkpoint, from the first evaluated, at update 1,500 and a drift of 2.50%, to the last, at 7.14%. The loss is led by ARC-Easy, which falls from 0.735 to 0.64–0.66. *tulu* and *smoltalk* forget nothing at any checkpoint up to a drift of 8.97%: their forgetting lies between −1.74 and −0.30%, inside the floor, and their MMLU rises from 0.383 to 0.418–0.439. At matched drift the range of forgetting across recipes is 5.0 pp at the 3% mark and 5.8 pp at the 6% mark, against a floor of 2.2 pp. The total drift gives $r = -0.04$ against forgetting over the 12. The IFEval gain is likewise a property of the recipe: +12.0 to +15.5 points for *tulu*, +10.5 to +16.5 for *smoltalk* and 0.0 to +3.5 for *mathcot*. The GSM8K score of *mathcot* cannot be read, because the 256-token cap cuts its reasoning chains short.

The dose is partly repetition. When a new pass over the training subset begins, the drift jumps within 20 to 40 updates while the training loss falls (Fig. 4a): *tulu* by 0.51 pp at the second pass and by 0.91 and then 0.73 pp at the third, from 4.06 to 5.70%; *smoltalk* by 1.88 pp at the third, from 6.19 to 8.07%. *mathcot* drifts smoothly. Both marks of *tulu* were reached just after a pass boundary. No benchmark moved with these jumps. The repeated examples are being memorised, a process whose capacity Morris et al. [14] estimate at about 3.6 bits per parameter, and the memorisation lengthens the code on fresh text by up to 1.9 pp without costing general accuracy.

One exploratory reading examined the shape of the drift across corpora. Its hypothesis, stated before it was computed but after the benchmarks were seen, was that *mathcot* drifts least on arXiv and most on Gutenberg; it was not supported. The forgetting recipe does drift least on Wikipedia, at 0.70 and 0.82 of its fresh mean at the two marks against 0.97 to 1.09 for the other recipes. This is a lead and not a result.

5. Discussion

5.1 Three practical rules

For post-training of models of this size the measurements support three rules.

1. *Evaluate the first checkpoint of a new recipe.* The 4% loss of *mathcot* was complete at the first checkpoint evaluated, after 1,500 updates and a drift of 2.5%, and did not grow over the following 1,964 updates. One benchmark pass at that point would have found it. The drift did not distinguish the recipe at any dose.
2. *Conversational SFT and DPO were safe to large doses.* Our *tulu* and *smoltalk* runs forgot nothing up to a drift of 9%, and the public OLMo-2 chain of SFT, DPO and RLVR forgot nothing up to 14.7%. The exception is SmolLM2-1.7B-Instruct, which lost 3.8%, mostly on ARC, at 3.6%. Our SmolTalk run on OLMo-2 did not reproduce the loss, which places its cause in the base, the DPO stage or details of the official recipe rather than in the SmolTalk data alone.
3. *Treat a drift above 20% as an alarm.* Both checkpoints above it forgot heavily, 13.9 and 21.6%. The alarm is one-sided and its blind spots are large. Below 20% the drift did not indicate forgetting: Qwen3-1.7B forgot 4.7% at 16.2%, SmolLM2-Instruct 3.8% at 3.6% and *mathcot* 4.1 to 4.4% at 2.5 to 7.1%, while OLMo-2-DPO and -RLVR forgot nothing at 14 to 15%. A threshold at 15% flags the three public checkpoints above it, all of which forgot, and raises no false alarm, but it misses SmolLM2-Instruct and was placed after the data were seen.

The drift costs at least 67 times less than a benchmark pass and can be logged at every checkpoint, as we did every 20 updates. It does not replace the first benchmark pass of a recipe.

5.2 Relation to prior work

Huang et al. [7] excluded fine-tuned models from the relation between compression and benchmark scores. Our measurements support the exclusion: after post-training the code length on general text measures a distance from the pretrained model and not a capability. Information Capacity [24] compares base and post-trained models in a figure and reports that advanced reinforcement learning degrades compression more. In our checkpoints the reinforcement-learning steps themselves moved the code length least, and the large increases came with distillation and mathematical training. Yang [23] measured the increase in loss for 14 base and derived pairs on web text, two of them ours, and our temperature result on text written after the cutoff agrees with that study's recovered share of at most 36%.

Kalajdzievski [9] used the cross-entropy to the base model's predictions on WikiText-103 as the measure of forgetting itself. We tested whether a quantity of this kind predicts benchmark forgetting and found that it does so only above a threshold. Shenfeld et al. [20] found that the forward KL divergence from the base, measured on the inputs of the new task, predicts forgetting across SFT and RL. Our quantity is measured on general text and did not predict forgetting within a recipe family; the two findings concern different distributions and do not conflict. Harmon et al. [6] found low to moderate benchmark forgetting after reasoning training from base models; in our set the distillation of reasoning traces onto a mathematical base forgot 9.0% on its own. Jin and Ren [8] predicted which upstream examples a fine-tune forgets, from the increase of their log-perplexity; our measure is the aggregate over unseen text. Marek et al. [13] showed that a KL penalty on pretraining or self-generated text during fine-tuning nearly removes forgetting. Our results show that the distance is a poor gauge of forgetting at small doses; they say nothing against using it as a regulariser, which we have not tested. InstructGPT [16] chose its SFT learning rate partly by a low loss on pretraining validation data. Our data suggest that such a loss tells the size of the step taken and not what the step cost. Kostelec and Guo [10] showed that scoring by log-likelihood can understate the gap between a distilled model and its teacher. Our general benchmarks are scored by log-likelihood, so a loss that appears only in generation would be missed.

5.3 Limitations

The conclusions hold within the following limits of the measurement.

1. All models have between 0.5 and 3 billion parameters.
2. The benchmarks are subsampled (HellaSwag 2,000 documents, MMLU 40 questions per subject, GSM8K 300, IFEval 200). The noise floor of one comparison is therefore about ±2% of the base score, or about 1.3 points of accuracy, and differences in forgetting of about 1%, such as those between *tulu* and *smoltalk*, lie inside it.
3. The gain is confounded by the prompt format and the generation length. Chat-tuned models fail prompts written for base models, and the 256-token cap truncates long reasoning, so the mathematical gain of *mathcot* and of R1-Distill is unmeasured.
4. The second stage of our runs was a warm restart with a fresh optimiser, and its checkpoints carry a second warm-up in their dose. Every 6% mark and two of the three 3% marks come from that stage.
5. The numbers of checkpoints are small: 12 public ones from different laboratories, families and recipes, and 12 of our own from one base and one seed. The intervals cover the sample of text, not the variation between training runs.
6. The corpora are small, about 100 KB each. The Gutenberg corpus consists of new transcriptions of old books, and most of the Wikipedia corpus is new articles rather than reports of 2026 events.
7. Our recipes repeated their data up to four times, and the repetition moved the drift by up to 1.9 pp without moving any benchmark.

As the doses become smaller and the recipes more alike, the differences in forgetting become comparable with the benchmark noise and depend more critically on the recipe and on the text that was scored.

5.4 What would change the conclusions

Four measurements would test these conclusions where they are weakest. The recipe test repeated at 7 billion parameters would show whether forgetting remains a step at a scale where most post-training is done. Full benchmarks without subsampling would lower the noise floor below the differences of about 1% between the conversational recipes. Single-pass recipes would remove the repetition from the dose. A generation cap long enough for reasoning chains, 2,048 tokens or more on GSM8K, would measure the gain of the knowledge-heavy recipe and settle whether the drift is one-sided for it. A recipe whose forgetting grew steadily with its drift, at fixed recipe and base, would overturn the reading of forgetting as a step.

6. Summary

We measured the code length of post-trained language models on text written after their release, relative to their base models. Post-training lengthened the code at every stage, by about 0.1% for pure reinforcement learning, 1 to 7% per step of SFT or DPO, and 16 to 43% for distillation and mathematical training, and one temperature removes at most 35% of the increase. The increase is a dose of post-training. It tracks the instruction-following gain, and above 15% it was always accompanied by forgetting. Under recipe control, forgetting was a step set by the recipe by the first checkpoint, independent of the drift and of its direction across token classes. As a cheap alarm for large departures the drift is useful; as a meter of forgetting at the doses of ordinary instruction tuning it is not.

Acknowledgements

The experiments were implemented and executed with AI coding agents (Claude, by Anthropic) working under the author's direction. The designs, pre-registrations and interpretations were reviewed by the author. Training and benchmark runs used the GPU cluster of Aalborg University.

References

[1] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, et al. SmolLM2: When smol goes big – data-centric training of a small language model. arXiv:2502.02737, 2025. URL https://arxiv.org/abs/2502.02737.

[2] T. Cover and R. King. A convergent gambling estimate of the entropy of English. *IEEE Transactions on Information Theory*, 24(4):413–421, 1978. doi: 10.1109/TIT.1978.1055912.

[3] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv:2501.12948, 2025. URL https://arxiv.org/abs/2501.12948. Journal reference on the arXiv page: Nature 645, 633–638 (2025).

[4] Grégoire Delétang, Anian Ruoss, Paul-Ambroise Duquenne, Elliot Catt, Tim Genewein, et al. Language modeling is compression. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2309.10668.

[5] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, et al. The language model evaluation harness. Zenodo, doi:10.5281/zenodo.12608602, 2024. URL https://zenodo.org/records/12608602. Zenodo record of v0.4.3, the citation given in the project README; this report used version 0.4.13.

[6] Jackson Harmon, Andreas Hochlehnert, Matthias Bethge, and Ameya Prabhu. Mapping post-training forgetting in language models at scale. arXiv:2510.17776, 2025. URL https://arxiv.org/abs/2510.17776.

[7] Yuzhen Huang, Jinghan Zhang, Zifei Shan, and Junxian He. Compression represents intelligence linearly. In *Conference on Language Modeling (COLM)*, 2024. URL https://arxiv.org/abs/2404.09937.

[8] Xisen Jin and Xiang Ren. Demystifying language model forgetting with low-rank example associations. arXiv:2406.14026, 2024. URL https://arxiv.org/abs/2406.14026. NeurIPS 2025 according to the arXiv Comments field.

[9] Damjan Kalajdzievski. Scaling laws for forgetting when fine-tuning large language models. arXiv:2401.05605, 2024. URL https://arxiv.org/abs/2401.05605.

[10] Juan Gabriel Kostelec and Qinghai Guo. When perplexity lies: Generation-focused distillation of hybrid sequence models. arXiv:2603.26556, 2026. URL https://arxiv.org/abs/2603.26556.

[11] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, et al. Tülu 3: Pushing frontiers in open language model post-training. arXiv:2411.15124, 2024. URL https://arxiv.org/abs/2411.15124.

[12] Sanae Lotfi, Marc Finzi, Yilun Kuang, Tim G. J. Rudner, Micah Goldblum, and Andrew Gordon Wilson. Non-vacuous generalization bounds for large language models. In *International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2312.17173.

[13] Martin Marek, Dongkyu Cho, Shikai Qiu, Rumi Chunara, Pavel Izmailov, and Andrew Gordon Wilson. Forgetting in language models: Capacity, optimization, and self-generated replay. arXiv:2605.26097, 2026. URL https://arxiv.org/abs/2605.26097.

[14] John X. Morris, Chawin Sitawarin, Chuan Guo, Narine Kokhlikyan, G. Edward Suh, Alexander M. Rush, Kamalika Chaudhuri, and Saeed Mahloujifar. How much do language models memorize? arXiv:2505.24832, 2025. URL https://arxiv.org/abs/2505.24832.

[15] Open R1. OpenR1-Math-220k. Hugging Face dataset open-r1/OpenR1-Math-220k, Apache-2.0, 2025. URL https://huggingface.co/datasets/open-r1/OpenR1-Math-220k.

[16] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, et al. Training language models to follow instructions with human feedback. arXiv:2203.02155, 2022. URL https://arxiv.org/abs/2203.02155.

[17] Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, et al. Qwen2.5 technical report. arXiv:2412.15115, 2024. URL https://arxiv.org/abs/2412.15115.

[18] C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3–4): 379–423, 623–656, 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x.

[19] C. E. Shannon. Prediction and entropy of printed English. *Bell System Technical Journal*, 30(1): 50–64, 1951. doi: 10.1002/j.1538-7305.1951.tb01366.x.

[20] Idan Shenfeld, Jyothish Pari, and Pulkit Agrawal. RL's razor: Why online reinforcement learning forgets less. arXiv:2509.04259, 2025. URL https://arxiv.org/abs/2509.04259.

[21] Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, et al. 2 OLMo 2 furious. arXiv:2501.00656, 2024. URL https://arxiv.org/abs/2501.00656. A shorter version was accepted to COLM 2025 according to the arXiv Comments field.

[22] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, et al. Qwen3 technical report. arXiv:2505.09388, 2025. URL https://arxiv.org/abs/2505.09388.

[23] Hanyu Yang. Is forgetting in post-trained language models just a shift in vocabulary bias? Preprint on alphaXiv, 2026. URL https://www.alphaxiv.org/abs/2610.language-model-forgetting-vocabulary-bias. Submitted 12 October 2026; no arXiv identifier resolved on the access date, 16 October 2026.

[24] Cheng Yuan, Jiawei Shao, and Xuelong Li. Information capacity: Evaluating the efficiency of large language models via text compression. arXiv:2511.08066, 2025. URL https://arxiv.org/abs/2511.08066.

[25] Weihao Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. SimpleRL-Zoo: Investigating and taming zero reinforcement learning for open base models in the wild. In *Conference on Language Modeling (COLM)*, 2025. URL https://arxiv.org/abs/2503.18892.

A. Pre-registered expectations and outcomes

Table 7 lists every rule and expected value that was committed before its run, the observed value and the verdict, followed by the deviations from each plan. Where an experiment's readings were added after its run, they are marked as such in the text and are not listed here as tests.

Table 7. Pre-registered rules and expectations against the outcomes, and the deviations from each plan.

Exp.	Rule or expectation	Observed	Verdict
1	Measure the four pairs on fresh text in raw and chat conditions (no numeric expectation)	All 12 pair and corpus drifts positive, intervals above zero	reported
	Deviations. A document-start separator is scored before every window, so that every byte is coded; the planned news corpus was replaced by Gutenberg, because Wikinews has published nothing since May 2026.		
2	Both large-drift pairs recover ≥ 70%: confidence shift; both ≤ 30%: change of belief; else mixed	Qwen3 35%, R1-Distill 19%	mixed
	Deviations. The check of $T = 1$ against the first experiment (at most 10^{-4} per window) failed for four reference rows produced by a superseded fp16 conversion; rescoring them matched to 1.1×10^{-6}.		
3	E1: OLMo-2 SFT step +3 to +8% and the largest OLMo-2 step	+6.99%, larger than DPO by 0.34 points	met, narrowly
	E2: OLMo-2 DPO step +0 to +2%	+6.65%	not met
	E3: OLMo-2 RLVR step within ±1%	+0.49%	met
	E4, E5: SimpleRL-Zoo 0.5B and 1.5B +0.5 to +3%	+0.18%, +0.12%	not met
	Headline: an RL stage below its predecessor with its interval below zero	none on any corpus	not triggered
	Reproduction of the four laptop models within 10^{-3} bpb	within 5×10^{-5}	met
	Deviations. The job ran on a T4 rather than the planned GPU.		
4	Gate 1: $r(\Delta_{\text{fresh}}, \text{forgetting}) \geq 0.8$ pass, ≥ 0.6 soft	0.887 $[-0.01, 0.98]$	pass, on two points
	One-sidedness: $\lvert r(\Delta_{\text{fresh}}, \text{GSM8K gain})\rvert < 0.3$	+0.383	not shown
	E1: $0.6 \leq r < 0.8$	0.887	not met
	E2: r with the gain below 0.3	+0.383	not met
	E3: enwik8 $r \geq$ fresh $r - 0.05$	0.949 against 0.887	met
	E4: R1-Distill has the largest drift and lies below the fitted line	largest, 2.1 points above the line	not met
	Deviations. The chat-template gain was added after the main run and became the primary gain measure; the chat-tuned subset has 8 checkpoints, not the 9 stated; Harmon et al.'s sample-wise forgetting was not computed because samples were not logged; each checkpoint ran in one process, with the generation configuration of the checkpoint reset to token ids only.		
5	Fail rule: $r(\Delta_K, \Delta_S) > 0.9$ over the 12	0.925	triggered
	H1: knowledge drift ranks forgetting at least as well as total drift, and $r \geq 0.6$ without the math line	ρ 0.671 against 0.671; $r = 0.48$	fail
	H2: $r(\Delta_S, \text{gain}) \geq 0.8$ and $r(\Delta_K, \text{gain}) < 0.5$	0.805 and 0.844	fail
	H3: knowledge share of SmolLM2-Instruct above that of OLMo-2-DPO	63.0 [60.1, 66.2] against 44.7 [42.6, 46.8]	pass
	H4: ρ(enwik8 excess, MMLU loss) ≥ $\rho(\Delta_{\text{fresh}}$, MMLU loss)	0.42 against 0.34	pass, weak
	E1: the fail rule triggers; E2: the predicted class ordering of four checkpoints	both as predicted	met
	Deviations. Tagger artefacts were left uncorrected, as the definitions were fixed: LaTeX mathematics tagged as proper nouns, hyphen-led pieces tagged as punctuation; the frequency-only variant, free of both, gives the same verdicts.		

Exp.	Rule or expectation	Observed	Verdict
6	H1: $r(K-S, \text{forgetting}) \geq 0.8$ pass, ≥ 0.6 soft, over the marks	0.431	fail
	H2: $\lvert r(K-S, \text{IFEval gain})\rvert < 0.3$	−0.405	fail
	H3: forgetting range across recipes above the floor at ≥ 2 marks	5.04 and 5.83 against 2.22 pp	pass
	H4: contrast and forgetting ordered *mathcot* > *smoltalk* > *tulu* at 6%	contrast 3.41, 2.38, 3.37	fail
	Fail rule: $r(K-S, \Delta_{\text{fresh}}) > 0.9$, or H1 < 0.6 with the H4 order violated	H1 0.43, order violated	triggered
	E0: every recipe reaches 6% within 1,500 updates	none did	not met
	E1: H1 at least soft and H2 pass; E2: the fail rule does not trigger	neither	not met
	At 6%, forgetting: *tulu* 0 ± 2, *smoltalk* 2 to 4, *mathcot* 3 to 8	−1.74, −0.89, +4.10	met, not met, met
	At 6%, contrast: *tulu* −3 to +1, *smoltalk* +2 to +5, *mathcot* +5 to +12	+3.37, +2.38, +3.41	not met, met, not met
	At 6%, IFEval gain ordered *tulu* > *smoltalk* > *mathcot*	13.5, 13.5, 0.0	tie, then met
	Deviations. The wall-time cap was raised from 5 to 8.5 hours after the dry run measured 15.4 s per update; the second stage was registered before any of its updates; the 10% mark was never reached, so H3 was scored on two marks and H1 and H2 on six checkpoints, not nine; there are 12 checkpoints, not the 11 of the brief; the per-corpus hypothesis was stated after the benchmarks were seen and was not supported.		

B. Compute, software and reproduction

Table 8 gives the compute of each experiment, excluding dry runs; the cluster work totals about 59 GPU-hours.

Table 8. Compute per experiment.

Experiment	Hardware	Compute
1, raw and chat scoring	Apple M2 Pro, MPS, fp16	21 min for the main table, 11 min for the document-start ablation
2, temperature	Apple M2 Pro, MPS, fp16	25 min of scoring
3, staged checkpoints	1 Tesla T4, fp16	18 min 42 s for 14 models and 4 corpora (0.3 GPU-hours)
4, benchmarks	NVIDIA A10	6.2 GPU-hours for 18 checkpoints, 3.3 GPU-hours for the chat-template addendum
5, token classes	Apple M2 Pro, CPU	68 s, no new scoring
6, controlled recipes	NVIDIA A40 and A10	about 45 A40-hours of training (6.4 to 6.5 h per recipe in stage 1, about 8.6 h in stage 2), about 4 A10-hours of evaluation, 29 min of CPU for data preparation

Software. Python 3.12. On the laptop: PyTorch 2.14.1, transformers 5.19.0, NumPy 2.5.4, pandas 3.0.6 and Matplotlib 3.11.2. On the cluster: the NGC PyTorch 25.09 container (PyTorch 2.9.0a0), transformers 5.19.0, lm-evaluation-harness 0.4.13, datasets 5.1.0 and accelerate 1.15.0. For the token classes: spaCy 3.8.16 with `en_core_web_sm` 3.8.0 and wordfreq 3.1.1. Exact versions are pinned in each experiment's requirements file.

Repository and reproduction. The code, the dated corpora with their provenance files, the per-token bits, the benchmark outputs and this report are in `https://github.com/gottn/compression-intelligence`. Each experiment is rerun with `bash expNN_*/run_all.sh`, whose header lists the cluster steps. The figures are produced by `paper/figures/make_figures.py`, which reads the committed result tables, checks every plotted number against the experiments' RESULTS.md and fails if one disagrees. The report is rebuilt with `make -C paper`, and `./check.sh` runs the linter and the tests, including those of the figures.